Advanced Deep Learning Systems
Lab 4 — Software Performance Engineering

Comprehensive Technical Reference & Cheat Sheet

torch.compile · Kernel Fusion · FlashAttention · CUDA Kernels · MXINT Quantization

---

Section	Core Topics
1. torch.compile	JIT compilation pipeline, TorchDynamo, TorchInductor, AOT Autograd
2. GPU Architecture & Memory	Memory hierarchy, compute vs. memory bound, roofline model
3. Kernel Fusion	Motivation, Linear+ReLU example, bandwidth analysis
4. FlashAttention (SDPA)	Tiled attention, online softmax, IO complexity
5. CUDA Kernels	Execution model, threads/warps/blocks, occupancy, performance tuning
6. Quantization & MXINT	Fixed-point, MXINT format, effective bitwidth, dequantization
7. Custom Kernel Case Study	MXINT8 dequant kernel, CPU vs GPU, CUTLASS/CUTE
8. Model Quantization	Weight-only quantization, DeBERTa results, memory analysis
9. Performance Strategy Guide	Decision framework, when to apply which technique

1. torch.compile — Automatic Performance Tuning

1.1 Overview

torch.compile is PyTorch 2.0's flagship optimization API. It transforms a Python-level model definition into an optimized executable by capturing the computational graph, applying graph-level transformations (operator fusion, constant folding, memory planning), and generating hardware-specific low-level code. The interface is deliberately minimal — a single function call or decorator — while the underlying machinery is a sophisticated multi-stage compiler pipeline.

1.2 JIT Compilation — Conceptual Background

Just-In-Time compilation sits between pure interpretation (slow but flexible) and ahead-of-time compilation (fast but rigid). A JIT compiler monitors program execution at runtime, identifies performance-critical regions ("hot paths"), and selectively compiles them to native code. The key trade-off is compilation latency versus execution speedup: compilation must be fast enough that its cost is amortized over subsequent executions.

In PyTorch's context, JIT compilation addresses a fundamental tension: Python's dynamism makes models easy to write and debug, but incurs per-operation interpreter overhead, prevents cross-operation optimization, and prevents the compiler from seeing the "big picture" of the computation. torch.compile resolves this by capturing a static graph from dynamic Python code, optimizing it holistically, and caching the compiled result.

1.3 The Three Pillars

1.3.1 TorchDynamo — Graph Capture

TorchDynamo (`torch._dynamo`) intercepts Python bytecode execution via CPython's frame evaluation API (PEP 523). It traces PyTorch operations into an **FX graph** — a directed acyclic graph where nodes represent operations and edges represent tensor data flow. Unlike previous tracing approaches (torch.jit.trace, torch.fx.symbolic_trace), Dynamo handles arbitrary Python control flow by inserting *graph breaks*: when it encounters Python code that cannot be captured (e.g., data-dependent branches, external library calls), it splits the program into captured sub-graphs and un-optimized Python regions.

- Operates at CPython bytecode level — sees every operation the interpreter executes
- Produces FX graphs consisting of placeholder (input), call_function, call_method, and output nodes
- Installs **guards**: boolean conditions (e.g., 'input tensor has shape [B,3,224,224]') that, when violated, trigger re-compilation
- Graph breaks degrade performance but preserve correctness; minimizing them is a key optimization goal

1.3.2 TorchInductor — Code Generation

TorchInductor (`torch._inductor`) is the default compiler backend. It receives the FX graph, performs optimization passes (operator fusion, layout optimization, memory planning), and generates executable code. For GPU targets, it emits **OpenAI Triton** kernels — a Python-based GPU programming language that compiles to PTX/SASS. For CPU targets, it generates C++ code with OpenMP parallelism.

- Pattern-matches the FX graph to identify fusible operation sequences (e.g., pointwise chains, reductions)

- Uses Triton's auto-tuning to select optimal tile sizes, number of warps, and other kernel parameters
- Generates **persistent kernels** that process multiple tiles per thread, reducing kernel launch overhead
- Applies **loop tiling**, **vectorization**, and **operator scheduling** to maximize hardware utilization

1.3.3 AOT Autograd — Full-Graph Backward Capture

Ahead-of-Time Autograd traces both the forward and backward passes before execution. Normally, PyTorch records the autograd tape dynamically during forward execution. AOT Autograd instead generates a **joint forward-backward graph**, enabling the compiler to optimize across the forward-backward boundary — for instance, fusing a forward pointwise operation with its corresponding backward gradient computation, or recomputing cheap intermediate values rather than storing them (trading compute for memory).

Compilation Pipeline: Python source → **TorchDynamo** (bytecode interception → FX graph) → **AOT Autograd** (forward+backward joint graph) → **TorchInductor** (fusion + Triton/C++ codegen) → Cached native code

1.4 Performance Considerations

torch.compile is not a universal speed-up. Understanding when and why it helps — and when it does not — is critical for effective use.

- **Cold start overhead:** The first invocation incurs graph capture + compilation cost (often several seconds to minutes for large models). This is amortized only if the model runs many iterations.
- **Dynamic shapes:** If input shapes vary, Dynamo may insert graph breaks or trigger re-compilations. Use `torch.compile(dynamic=True)` for shape-polymorphic compilation, though this can reduce optimization quality.
- **Graph breaks:** Operations that cause graph breaks (e.g., print statements, unsupported Python constructs, data-dependent control flow) split the graph into smaller fragments, limiting fusion opportunities.
- **Small models:** If the model is already fast (e.g., a single linear layer), the overhead of graph capture and kernel launch may exceed the optimization benefit.
- **Compilation modes:** `mode='reduce-overhead'` uses CUDA graphs to eliminate kernel launch overhead; `mode='max-autotune'` tries more kernel configurations but takes longer to compile.

Benchmarking Tip: Always exclude warm-up iterations from timing. The lab's result showing the 'optimized' model being slower is likely due to compilation occurring during the timed region. A proper benchmark should: (1) run 1-2 warm-up iterations to trigger compilation, (2) synchronize the GPU with torch.cuda.synchronize(), (3) then time subsequent iterations.

1.5 Usage

```
# Simplest usage — wraps a model or function
opt_model = torch.compile(model)
# With compilation mode
opt_model = torch.compile(model, mode='reduce-overhead')
# As decorator
@torch.compile
def fused_gelu(x):
return x * 0.5 * (1.0 + torch.tanh(0.7978 * (x + 0.044715 * x**3)))
```

2. GPU Architecture & Memory Hierarchy

2.1 Why GPU Architecture Matters

Every performance optimization in this lab — kernel fusion, FlashAttention, custom CUDA kernels, quantization — is fundamentally motivated by the GPU's hardware characteristics. Understanding the memory hierarchy, execution model, and bottleneck analysis is prerequisite to reasoning about any optimization.

2.2 GPU Memory Hierarchy

Modern NVIDIA GPUs have a deep memory hierarchy. Performance-critical code must be designed around this hierarchy to minimize data movement through the slower levels.

Memory	Scope	Typical Size	Latency	Bandwidth
Registers	Per-thread	~256 KB / SM	0 cycles (operand)	Highest
Shared Mem / L1	Per-threadblock	48–228 KB / SM	~20-30 cycles	~10+ TB/s aggregate
L2 Cache	Entire GPU	4–96 MB	~200 cycles	~4-6 TB/s
Global (HBM)	Entire GPU	8–80 GB	~400-800 cycles	~1-3 TB/s
System (CPU)	Host	GBs–TBs	~us (PCIe/NVLink)	~30-100 GB/s

Core Principle: Global memory (HBM) has ~400-800 cycle latency vs. ~20-30 cycles for shared memory and ~0 cycles for registers. Every redundant round-trip to HBM is wasted time. The entire motivation for kernel fusion is eliminating these round-trips by keeping intermediate data on-chip.

2.3 Compute-Bound vs. Memory-Bound

An operation's performance bottleneck is determined by its **arithmetic intensity** — the ratio of floating-point operations (FLOPs) to bytes transferred from memory. This concept is formalized in the **Roofline Model**.

- **Memory-bound:** The operation's arithmetic intensity is below the hardware's "ridge point" (compute FLOPS / memory bandwidth). The GPU finishes computing before data arrives. Examples: element-wise ops (ReLU, GELU, add), reductions (softmax, LayerNorm), small matrix multiplications.
- **Compute-bound:** The operation's arithmetic intensity exceeds the ridge point. The GPU's compute units are fully saturated. Examples: large GEMM (matrix multiply), convolutions with large channels.

For a modern A100 GPU with ~2 TB/s HBM bandwidth and ~312 TFLOPS (FP16 Tensor Core), the ridge point is ~156 FLOPS/byte. Operations below this threshold are memory-bound. Most element-wise operations have an arithmetic intensity of ~0.25-1 FLOP/byte — they are severely memory-bound, making them excellent fusion candidates.

Practical Implication: ReLU (relu(x) = max(0, x)) performs 1 comparison per element but must load and store that element from/to global memory (8 bytes for FP32). Its arithmetic intensity is ~0.125 FLOPS/byte — over 1000x below the ridge point. ReLU's runtime is entirely dominated by memory access, not computation. This is why fusing ReLU with a preceding operation (which already has the data in registers) is so effective.

3. Kernel Fusion

3.1 What is a GPU Kernel?

A GPU **kernel** is a function that executes in parallel across thousands of GPU threads. When PyTorch calls `torch.relu(x)`, it dispatches a CUDA kernel where each thread computes max(0, x[i]) for its assigned element. A kernel launch involves: (1) copying launch parameters to the GPU, (2) scheduling threadblocks on streaming multiprocessors (SMs), (3) executing the kernel, (4) synchronizing. This process has a fixed overhead of roughly 5-10 microseconds per launch.

3.2 The Case for Fusion

When two operations are executed as separate kernels, the intermediate result must be written to global memory by the first kernel and read back by the second. Kernel fusion combines them into a single kernel, keeping intermediates in fast on-chip memory. The benefits are threefold:

- **Reduced memory traffic:** Eliminates read/write of intermediate tensors from/to HBM (often the dominant cost for memory-bound operations)
- **Fewer kernel launches:** Each launch has ~5-10 us overhead; fusing N operations saves (N-1) launch overheads
- **Register reuse:** Intermediate values stay in registers (0-cycle access) rather than being evicted to global memory

3.3 Detailed Example: Linear + ReLU

Consider a module that applies a linear transformation followed by ReLU. We trace the exact sequence of memory operations for both the unfused and fused cases.

Unfused — Two Separate Kernel Launches:

1	linear	Load x tiles from HBM to shared memory	HBM read: O(B x in_features)
2	linear	Load W tiles from HBM to shared memory	HBM read: O(in x out_features)
3	linear	Tile-level matmul, accumulate in registers	Compute (no memory access)
4	linear	Write y1 = xW + b to HBM	HBM write: O(B x out_features)
5	relu	Load y1 from HBM	HBM read: O(B x out_features)
6	relu	Compute max(0, y1) in registers	Compute (trivial)
7	relu	Write y2 to HBM	HBM write: O(B x out_features)

Fused — Single Kernel Launch:

1	linear+relu	Load x tiles from HBM to shared memory	HBM read: O(B x in_features)

2	linear+relu	Load W tiles from HBM to shared memory	HBM read: O(in x out_features)
3	linear+relu	Tile-level matmul, accumulate in registers	Compute (no memory access)
4	linear+relu	Apply max(0, result) in registers	No memory access (still in regs!)
5	linear+relu	Write y2 to HBM	HBM write: O(B x out_features)

Bandwidth Saved: Fusion eliminates 1 write + 1 read of the intermediate y1 tensor. For a batch of 128 with out_features=1024 in FP32, that is 2 x 128 x 1024 x 4 bytes = **1 MB saved per layer**. In a deep network with hundreds of such layers, this adds up to hundreds of MBs of eliminated memory traffic per forward pass.

3.4 When Fusion Helps Most

Not all operation pairs benefit equally from fusion. The benefit depends on the relative costs of compute and memory access for each operation.

GEMM + pointwise (ReLU, GELU, bias add)	**High**	Pointwise ops are purely memory-bound; fusion makes their cost essentially zero
Pointwise + pointwise (e.g., add + mul + exp)	**High**	Entire chain becomes a single kernel reading/writing each element once
GEMM + reduction (e.g., matmul + softmax)	**High**	Eliminates large intermediate matrix; this is what FlashAttention does
GEMM + GEMM	**Low**	Both are compute-bound; the memory traffic saving is small relative to compute time
Reduction + pointwise	**Medium**	Saves one intermediate buffer, but reduction itself has limited parallelism

4. Scaled Dot-Product Attention & FlashAttention

4.1 Standard SDPA

The Scaled Dot-Product Attention (SDPA) mechanism is the computational core of every Transformer. Given matrices Q (queries), K (keys), V (values) of shapes [B, H, N, d] where B=batch, H=heads, N=sequence length, d=head dimension:

$$\textbf{Attention}(Q, K, V) = \text{softmax}\left(Q K^T / \sqrt{d} \right) \cdot V$$

The naive implementation consists of three sequential operations: (1) $S = QK^T/\sqrt{d}$ — a GEMM producing the N x N attention score matrix, (2) $A = \text{softmax}(S)$ — a row-wise reduction and normalization, and (3) $O = AV$ — a GEMM producing the output. The critical problem is that the intermediate S matrix has shape [B, H, N, N] — for sequence length N=4096, this is 16M entries per head, consuming gigabytes of HBM for realistic batch sizes.

4.2 FlashAttention: Tiled Fused SDPA

FlashAttention (Dao et al., 2022) fuses the entire GEMM → softmax → GEMM pipeline into a single kernel by processing Q, K, V in tiles. It **never materializes the full N x N attention matrix** in HBM, instead computing and consuming tiles of it on-the-fly in SRAM.

4.2.1 The Online Softmax Algorithm

Standard softmax requires two passes: (1) find the row maximum m = max(S[i,:]) for numerical stability, (2) compute sum = sum(exp(S[i,:] - m)). This seems to require the entire row before producing any output. The **online softmax** trick maintains running statistics that are incrementally updated as new tiles arrive:

- Maintain running max **m** and running sum-of-exponentials **l**
- For each new tile of K: compute local scores, update $m_{new} = \max(m_{old}, \text{local\_max})$
- Apply correction factor: rescale previous partial output by $\exp(m_{old} - m_{new}) / l_{new}$
- Accumulate the new tile's contribution to the output
- This is **mathematically exact** — no approximation, no accuracy loss

4.2.2 IO Complexity Analysis

The key insight is that FlashAttention reduces **IO complexity** (bytes read/written to HBM) while keeping the same arithmetic complexity. This is significant because attention is memory-bound for typical head dimensions.

HBM reads	$O(N^2d + N^2)$	$O(N^2d^2 / M)$ where M = SRAM size
HBM writes	$O(N^2 + Nd)$	$O(Nd)$
Peak HBM usage	$O(N^2)$ for attention matrix	O(N) — only tiles in SRAM
Kernel launches	3+ (GEMM, softmax, GEMM, ...)	1 (fully fused)
Accuracy	Exact	Exact (online softmax is lossless)
Wall-clock speedup	Baseline	2-4x faster (memory-bound regime)

4.3 Performance Evaluation

FlashAttention's speedup increases with sequence length because the $O(N^2)$ memory savings become more impactful. At N=512, the attention matrix is small and fits in cache, yielding modest gains. At N=4096+, the naive approach spills to HBM, and FlashAttention provides 2-4x wall-clock speedup. Beyond performance, FlashAttention's O(N) memory enables training with sequence lengths that would otherwise cause out-of-memory errors.

PyTorch 2.0+ provides `F.scaled_dot_product_attention`, which automatically selects the best available backend: FlashAttention v2 (requires SM 80+, fp16/bf16), Memory-Efficient Attention (xFormers, broader hardware support), or a mathematical C++ fallback.

5

CUDA Kernels — Architecture & Performance

5.1 What is a CUDA Kernel?

A CUDA kernel is a C/C++ function that runs on an NVIDIA GPU. Unlike CPU functions that execute a single thread (or a small number of threads), a CUDA kernel launches thousands to millions of threads simultaneously. Each thread typically processes one or a few data elements. The programming model is **SIMT** (Single Instruction, Multiple Threads): groups of 32 threads called **warps** execute the same instruction in lockstep, with each thread operating on different data.

When PyTorch executes `torch.relu(x)`, it dispatches a CUDA kernel where each thread computes `max(0, x[i])` for its assigned element. A kernel launch involves: (1) copying launch parameters to the GPU, (2) scheduling threadblocks on streaming multiprocessors (SMs), (3) executing the kernel, and (4) synchronizing. This sequence has a fixed overhead of roughly **5–10 microseconds** per launch, regardless of kernel size.

Example: A Minimal CUDA Kernel

This kernel adds two vectors element-by-element. Each thread handles one element.

```
__global__ void vecAdd(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
// Launch with 256 threads per block
int blocks = (n + 255) / 256;
vecAdd<<<blocks, 256>>>(a, b, c, n);
```

The expression `blockIdx.x * blockDim.x + threadIdx.x` computes a globally unique thread index. `blockIdx.x` identifies which block the thread belongs to, `blockDim.x` is the number of threads per block, and `threadIdx.x` is the thread's position within its block. This pattern—computing a global index from block and thread IDs—is the fundamental idiom of CUDA programming.

5.2 The CUDA Execution Hierarchy

CUDA organizes threads into a three-level hierarchy. Understanding this hierarchy is essential for writing efficient kernels and reasoning about performance.

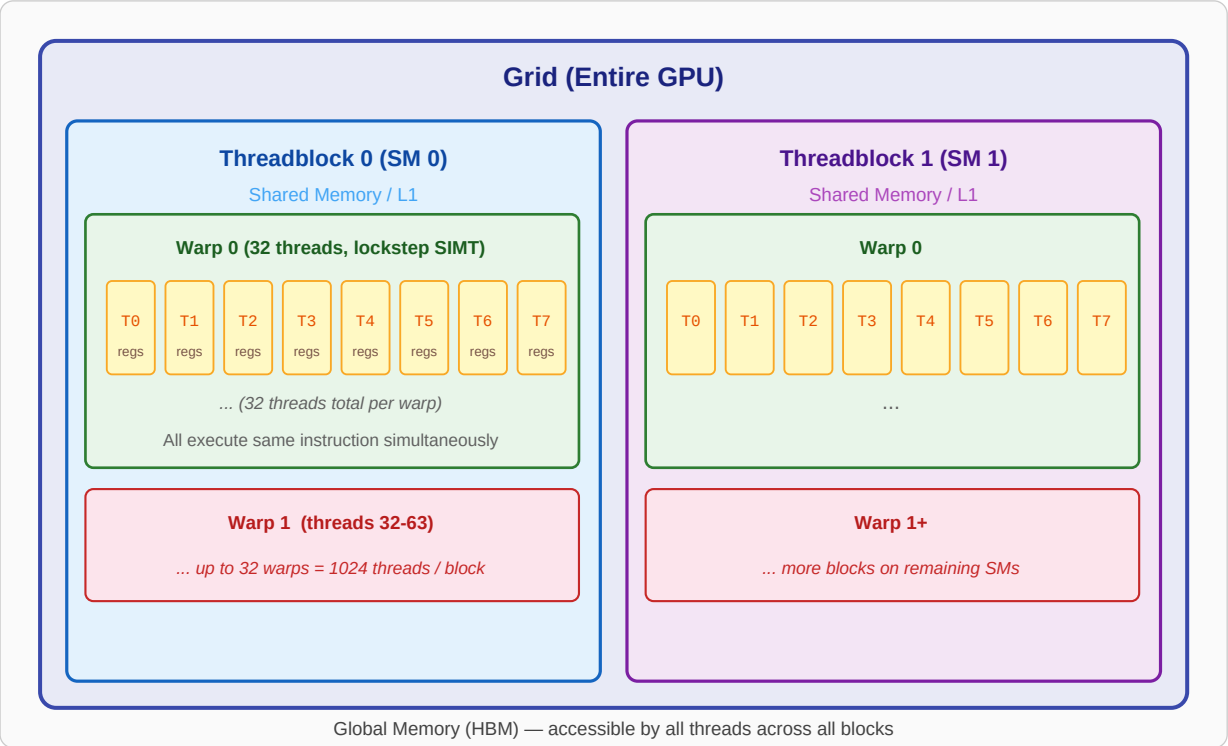

Figure 5.1: CUDA thread hierarchy—Grid, Threadblocks, Warps, and Threads

Level	Size	Hardware Mapping	Shared Resources
Thread	Smallest unit	CUDA core (ALU)	Private registers, local memory
Warp	32 threads	Warp scheduler on SM	Instruction stream (SIMT); warp-level primitives
Threadblock (CTA)	Up to 1024 threads (32 warps max)	One Streaming Multiprocessor (SM)	Shared memory, L1 cache, synchronization barriers
Grid	Up to millions of threads	Entire GPU (all SMs)	Global memory, L2 cache

> **Key Concept: The Warp is the Fundamental Scheduling Unit**
>
> The GPU does not schedule individual threads. It schedules **warps** of 32 threads. All 32 threads in a warp execute the same instruction at the same time (SIMT). If one thread in a warp stalls (e.g., waiting for memory), the **entire warp** stalls—but the warp scheduler immediately switches to another ready warp on the same SM. This is the primary mechanism for **latency hiding**.

5.3 Kernel Launch: What Actually Happens

When PyTorch (or any CUDA application) launches a kernel, the following sequence occurs:

- **Step 1 — Parameter Copy:** The CPU-side CUDA runtime copies kernel launch parameters (grid dimensions, block dimensions, function pointer, arguments) to the GPU via the command queue.
- **Step 2 — Block Distribution:** The GPU's **Gigathread Engine** distributes threadblocks to available SMs in a round-robin fashion. Each SM may host multiple threadblocks concurrently.

• **Step 3 — Warp Scheduling:** Each SM's warp scheduler partitions the threadblock into warps (groups of 32 threads) and begins scheduling them for execution.

• **Step 4 — Execution with Latency Hiding:** Warps execute instructions. When a warp stalls (e.g., waiting for global memory), the scheduler switches to another ready warp with zero overhead.

• **Step 5 — Completion:** When all threads complete, results are visible in global memory after synchronization (`cudaDeviceSynchronize()` or `torch.cuda.synchronize()`).

> **Kernel Launch Overhead**
>
> Steps 1–2 take approximately **5–10 µs** regardless of the kernel's workload. For tiny operations (e.g., ReLU on a tensor with 1024 elements), this overhead can exceed the actual compute time. This is a key motivation for **kernel fusion**: combining N operations into one kernel eliminates (N–1) launch overheads.

5.4 CUDA Performance Optimization

5.4.1 Occupancy

Occupancy is the ratio of active warps to the maximum warps an SM can support. Higher occupancy generally enables better latency hiding: when one warp stalls on a memory access, the SM switches to another active warp. Occupancy is limited by three resources per SM:

• **Registers per thread:** If each thread uses many registers, fewer threads (and thus fewer warps) fit on the SM. For example, if an SM has 65,536 registers and your kernel uses 128 registers per thread, you can fit at most 512 threads = 16 warps. If the SM supports 64 warps, occupancy is 16/64 = 25%.

• **Shared memory per threadblock:** If a threadblock requests too much shared memory, fewer threadblocks fit on each SM.

• **Maximum threadblocks per SM:** Hardware imposes a hard limit (typically 16–32 blocks per SM).

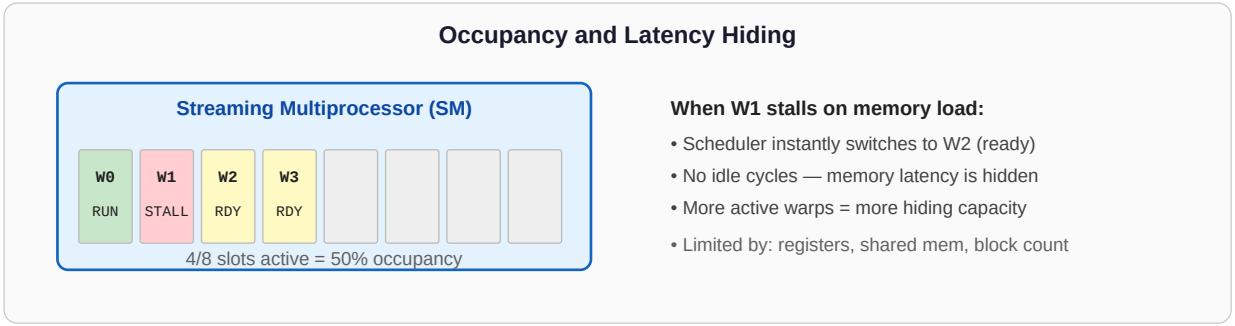

Figure 5.2: Occupancy enables latency hiding by switching between warps

Example: Computing Occupancy

Consider an SM that supports max 64 warps (2048 threads) and has 65,536 registers.
Kernel A uses 32 registers/thread, block size 256 (8 warps):
• Regs per block = 256 × 32 = 8,192 → can fit 65,536/8,192 = 8 blocks
• Warps = 8 blocks × 8 warps = 64 warps → **100% occupancy**
Kernel B uses 128 registers/thread, block size 256:
• Regs per block = 256 × 128 = 32,768 → can fit 65,536/32,768 = 2 blocks
• Warps = 2 blocks × 8 warps = 16 warps → **25% occupancy**
Kernel B has 4× less latency-hiding capacity. If it is memory-bound, it will be significantly slower than Kernel A.

5.4.2 Memory Coalescing

Global memory (HBM) is accessed in **transactions** of 32 or 128 bytes. When all 32 threads in a warp access contiguous, aligned memory addresses, these accesses are **coalesced** into a minimal number of transactions. Non-coalesced (scattered) accesses waste bandwidth by loading cache lines that contain mostly unused bytes.

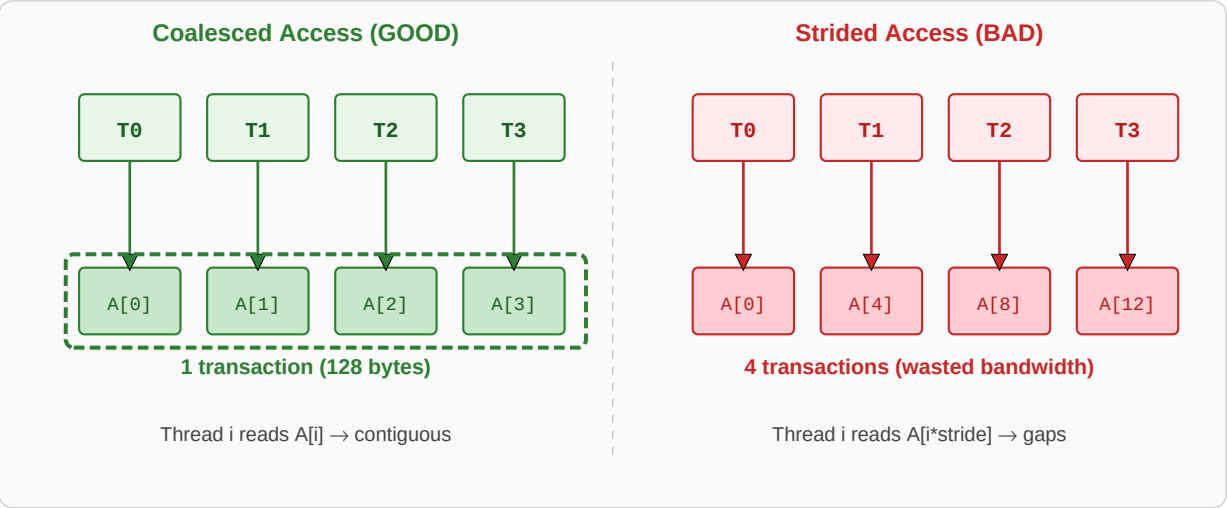

Figure 5.3: Coalesced vs. strided memory access patterns

Example: Row-Major vs. Column-Major Access

Consider a 2D matrix A[M][N] stored in row-major order (C-style). Threads in a warp processing one row each, accessing column j:
• **Row access** (A[row][0], A[row][1], ...): Thread i accesses A[row][i]. Elements are contiguous in memory → **coalesced**.
• **Column access** (A[0][col], A[1][col], ...): Thread i accesses A[i][col]. Elements are N positions apart in memory → **stride-N access, not coalesced**. Each thread triggers a separate cache line load.
Fix: Stage data through shared memory. Load a tile with coalesced access, then transpose in shared memory for column-oriented processing.

5.4.3 Bank Conflicts in Shared Memory

Shared memory is divided into 32 **banks** (one per warp lane). Consecutive 4-byte words map to consecutive banks. If two threads in a warp access different addresses in the **same bank**, these accesses are serialized—a **bank conflict**. A k-way bank conflict means the access takes k cycles instead of 1.

Example: Bank Conflict and Padding Fix

Shared memory array: `__shared__ float smem[32][32];`
If thread i accesses `smem[i][0]`, all threads access bank 0 → 32-way bank conflict (32× slower).
Fix with padding: Declare `__shared__ float smem[32][33];`
The extra column shifts each row by one bank position, so thread i accessing `smem[i][0]` now hits bank i → no conflicts.

5.4.4 Warp Divergence

When threads within a warp take different branches of an `if/else` statement, the warp must execute **both paths sequentially**, with inactive threads masked off. This is **warp divergence**, and it effectively halves (or worse) throughput.

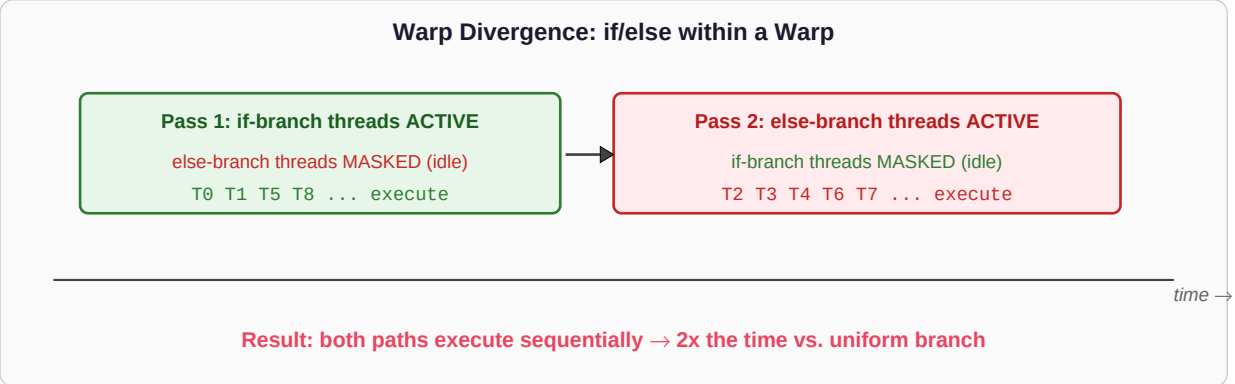

Figure 5.4: Warp divergence forces serial execution of both branches

Example: Divergent vs. Non-Divergent Code

Divergent (threads in same warp take different paths):

```
if (threadIdx.x % 2 == 0)   // even threads: path A
    result = expensive_A(x);
else                         // odd threads: path B
    result = expensive_B(x);
// Warp executes A then B = 2x time
```

Non-divergent (reorganize so entire warps take same path):

```
if (threadIdx.x < 16)  // first 16 threads = warp 0 (all take A)
    result = expensive_A(x);
else                   // next 16 threads = warp 1 (all take B)
    result = expensive_B(x);
// Each warp executes only its path = 1x time
```

5.4.5 Instruction-Level Parallelism (ILP)

Modern GPUs have deep pipelines. A single thread can have multiple independent instructions in-flight simultaneously. By unrolling loops and ensuring each thread has multiple independent

operations, the compiler can overlap arithmetic with memory access, hiding latency even at lower occupancy.

Example: Loop Unrolling for ILP

Instead of processing one element per iteration, process four. The four loads are independent, so the hardware can issue them simultaneously and hide memory latency:

```
// Low ILP: one load per iteration
for (int i = tid; i < n; i += stride)
    out[i] = in[i] * scale;
// High ILP: four independent loads per iteration
for (int i = tid; i < n/4; i += stride) {
    float a = in[4*i+0], b = in[4*i+1];
    float c = in[4*i+2], d = in[4*i+3];
    out[4*i+0] = a*scale; out[4*i+1] = b*scale;
    out[4*i+2] = c*scale; out[4*i+3] = d*scale;
}
```

5.5 Performance Checklist

Factor	What to Check	Impact if Wrong
Memory coalescing	Adjacent threads access adjacent addresses?	10–100× bandwidth waste
Occupancy	Enough active warps for latency hiding?	2–4× throughput loss
Shared memory bank conflicts	Access pattern avoids same-bank collisions?	Up to 32× shared mem slowdown
Warp divergence	All threads in warp take same branch?	Up to N× slowdown for N branches
Kernel launch overhead	Kernel does enough work to justify launch?	Launch cost dominates for tiny ops
Register spilling	Kernel uses fewer regs than SM limit?	Spills to slow local memory (DRAM speed)
Data reuse	Tiles reused from shared mem, not reloaded?	Redundant HBM traffic

6

Quantization & the MXINT Format

6.1 Why Quantize?

Neural network weights are typically stored in FP32 (32 bits per value) or FP16/BF16 (16 bits). A model with 7 billion parameters in FP32 requires 28 GB of storage—already exceeding the memory capacity of many GPUs. **Quantization** reduces numerical precision, converting FP32/FP16 values to lower bitwidths (INT8, INT4, etc.), thereby reducing memory footprint, bandwidth consumption, and (with appropriate hardware) computation cost.

The trade-off is straightforward: fewer bits means fewer representable values, which introduces quantization error. The goal of quantization research is to minimize this error while maximizing compression.

Approach	What is Quantized	Primary Benefit	Runtime Behavior
Weight-only	Model weights only	GPU memory savings (smaller weight storage)	Weights dequantized to FP16/BF16 before compute; activations in FP16
Weight + Activation	Weights and activations	Memory + compute savings (cheaper arithmetic units)	Low-precision matmul directly; requires calibration or QAT

6.2 Fixed-Point Numbers: The Building Block

Before understanding MXINT, you need to understand **fixed-point representation**. In a fixed-point number, the position of the radix (binary) point is **fixed** at a predetermined location, unlike floating-point where it "floats" via an exponent.

Consider a 4-bit unsigned fixed-point number with format `0b.XXXX`. The radix point is immediately before all 4 bits, so each bit represents a negative power of 2. The value is: $b_3 \times 2^{-1} + b_2 \times 2^{-2} + b_1 \times 2^{-3} + b_0 \times 2^{-4}$.

> **Example: 4-bit Unsigned Fixed-Point (format 0b.XXXX)**
>
> Each bit position has a fixed weight: 0.5, 0.25, 0.125, 0.0625
> • `0b.0000` = 0.0
> • `0b.0001` = 0.0625
> • `0b.0100` = 0.25
> • `0b.1000` = 0.5
> • `0b.1100` = 0.5 + 0.25 = 0.75
> • `0b.1111` = 0.5 + 0.25 + 0.125 + 0.0625 = 0.9375
> Range: [0, 0.9375] with step size 0.0625. Only **16 distinct values** in a narrow range.

> **The Problem with Pure Fixed-Point**
>
> Fixed-point's narrow dynamic range is problematic for neural networks. Weights across different layers can span several orders of magnitude (e.g., 0.001 to 10.0). A 4-bit fixed-point format with step 0.0625 cannot represent fine-grained differences near zero or large values simultaneously. This is exactly what MXINT addresses.

6.3 MXINT: Block-Scaled Integer Format

MXINT (Microscaling Integer), proposed by Microsoft and standardized by the Open Compute Project (OCP), solves fixed-point's range limitation through a simple but powerful idea: **share a single floating-point exponent across a group of fixed-point mantissas**. This provides the dynamic range of floating-point with the storage efficiency of integers.

The core intuition is that neighboring weights in a tensor tend to have similar magnitudes. By grouping them and assigning one shared scale factor (exponent), the per-element storage is kept low while the group can collectively represent a wide range of values.

6.3.1 Format Structure

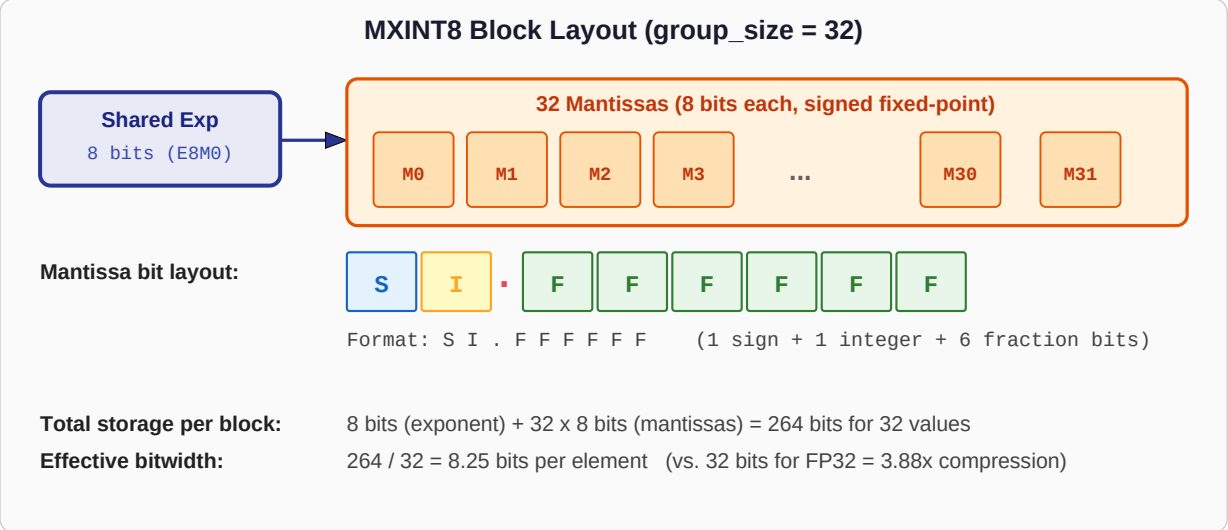

Figure 6.1: MXINT8 block layout with shared exponent and individual mantissas

An MXINT block consists of two components:

- **Shared exponent (8 bits):** An E8M0 format value (8-bit biased exponent with no mantissa). The bias is 127, so the scale factor is $2^{(exp-127)}$. This sets the "magnitude window" for the entire group.
- **G mantissas (k bits each):** Signed fixed-point values in the format `SI.FFFFFF` for MXINT8 (1 sign bit, 1 integer bit, 6 fraction bits). Each mantissa is independently signed, allowing positive and negative values within the same group.

Key Insight: How MXINT Achieves Wide Dynamic Range

The shared exponent acts like a **zoom level**. If a group of weights has large values (e.g., around 8.0), the exponent is set high (e.g., 130 → scale = 2^{3} = 8). If a group has small values (e.g., around 0.01), the exponent is set low (e.g., 120 → scale = 2^{-7} ≈ 0.0078). The mantissas then provide **relative precision** within that window. This is analogous to scientific notation: 6.022×10^{23} uses a mantissa (6.022) and an exponent (23) to represent a very large number efficiently.

6.3.2 MXINT8 Mantissa in Detail

The MXINT8 mantissa is an 8-bit value with the format `SI.FFFFFF`:

- **Bit 7 (S):** Sign bit. 0 = positive, 1 = negative.
- **Bit 6 (I):** Integer bit. Represents the value 1.0 (i.e., 2^{0}).
- **Bits 5–0 (FFFFFF):** Fractional bits. Represent 2^{-1}=0.5, 2^{-2}=0.25, ..., 2^{-6}=0.015625.

The magnitude of the mantissa (ignoring sign) ranges from 0.0 (`0.000000`) to 1.984375 (`1.111111`). There are 128 distinct unsigned mantissa values, and with the sign bit, 256 total (including positive zero and negative zero).

Example: Interpreting MXINT8 Mantissa Bytes

`0b 01.000000` = +1.0 (sign=0, integer bit=1, fraction=0)
`0b 11.000000` = -1.0 (sign=1, integer bit=1, fraction=0)
`0b 00.100000` = +0.5 (sign=0, integer bit=0, fraction=0.5)
`0b 01.100000` = +1.5 (sign=0, integer=1, fraction=0.5)
`0b 00.010000` = +0.25 (sign=0, integer=0, fraction=0.25)
`0b 01.111111` = +1.984375 (maximum positive value)
`0b 00.000001` = +0.015625 (minimum positive nonzero value)

6.3.3 Dequantization: MXINT to Float

Dequantization converts an MXINT value back to a standard floating-point number. The formula is:

$$\texttt{float\_value} = \texttt{mantissa\_fixed\_point} \times 2^{(\texttt{exponent} - 127)}$$

The exponent is stored with a bias of 127 (identical to IEEE 754 single-precision). An exponent byte of 127 means scale = 2^{0} = 1 (no scaling). An exponent of 130 means scale = 2^{3} = 8. An exponent of 120 means scale = 2^{-7} ≈ 0.0078.

Worked Example: Full MXINT8 Dequantization

Suppose we have a group with **shared exponent = 130** (scale = 2^3 = 8) and the following mantissas:
Mantissa 1: `0b 01.000000` = +1.0
→ float = +1.0 × 8 = **+8.0**
Mantissa 2: `0b 00.100000` = +0.5
→ float = +0.5 × 8 = **+4.0**
Mantissa 3: `0b 11.010000` = -(1.0 + 0.25) = -1.25
→ float = -1.25 × 8 = **-10.0**
Mantissa 4: `0b 00.000001` = +0.015625
→ float = +0.015625 × 8 = **+0.125**
All four values share the same exponent but represent a range from -10.0 to +8.0.

6.3.4 Effective Bitwidth & Storage Efficiency

The 8-bit shared exponent is a per-group cost that gets amortized over all mantissas in the group. The **effective bitwidth** per element is:

```
effective_bits = mantissa_bits + 8 / group_size
```

Format	Mantissa Bits	Group Size	Effective Bits	Compression vs FP32
MXINT4 (g=4)	4	4	6.0	5.3×
MXINT4 (g=32)	4	32	4.25	7.5×
MXINT8 (g=32)	8	32	8.25	3.88×
Pure INT4	4	N/A	4.0	8.0×
Pure INT8	8	N/A	8.0	4.0×
FP16 / BF16	—	N/A	16.0	2.0×
FP32	—	N/A	32.0	1.0× (baseline)

The Group Size Trade-off

Larger groups (e.g., 128) reduce the effective bitwidth (cheaper storage) but force more values to share one exponent. If a group contains both very large and very small weights, the small weights lose precision because the exponent is set to accommodate the large ones. **Smaller groups** (e.g., 4) provide better per-group dynamic range but waste more bits on exponent overhead. **Group size 32** is the recommended default—a practical sweet spot validated by extensive benchmarks.

Example: Why Group Size Matters

Consider 4 weights: [0.001, 0.002, 0.001, 5.0] with MXINT8 (g=4).
The exponent must accommodate 5.0, so scale ≈ 4.0 (exp=129). The mantissa step at this scale is 0.015625 × 4 = 0.0625.
• 5.0 → mantissa 1.25, dequant = 5.0 ✔ (exact)
• 0.001 → nearest mantissa 0.0 or 0.015625 → dequant = 0.0 or 0.0625
The true value 0.001 is **lost**—quantization error is 62× the original.
If these 4 values were in **separate groups**, the small values would get a low exponent (e.g., exp=117, scale=2^{-10}=0.000977) and be represented accurately.

6.4 MXINT8 for Custom Hardware Acceleration

When **both** activations and weights are quantized to MXINT8, custom hardware can exploit the format by performing the core matrix multiplication on **8-bit integer mantissas**—dramatically cheaper than floating-point multiply. The shared exponents are handled separately.

For a dot product of two MXINT8 groups (one from weights, one from activations), the computation decomposes as: (1) compute the integer dot product of the two mantissa vectors (cheap integer MACs), and (2) multiply the result by the product of the two shared exponents (a single floating-point multiply). Since the vast majority of operations are integer, this enables smaller, lower-power multiply-accumulate (MAC) units.

Example: MXINT8 Dot Product Decomposition

Weight group: exponent = 130, mantissas = [1.0, 0.5, -0.25, 1.5]
Activation group: exponent = 128, mantissas = [0.5, 1.0, 0.5, -1.0]
Step 1 — Integer mantissa dot product:
1.0×0.5 + 0.5×1.0 + (-0.25)×0.5 + 1.5×(-1.0) = 0.5 + 0.5 - 0.125 - 1.5 = -0.625
(These are fixed-point multiplies, implementable as integer ops with fixed shift)
Step 2 — Scale by exponents:
scale = $2^{(130-127)} \times 2^{(128-127)} = 8 \times 2 = 16$
Result: -0.625 × 16 = **-10.0**
An 8-bit integer multiplier uses ~6× less area and ~6× less energy than FP32.

6.5 Comparison to Other Quantization Formats

Format	Per-Element Bits	Dynamic Range	Hardware Support	Use Case
MXINT8 (g=32)	8.25 effective	Wide (shared exp)	OCP standard; custom accelerators	Weight-only or weight+activation
INT8 (symmetric)	8	Fixed [-127, 127] × scale	NVIDIA Tensor Cores (INT8 mode)	Inference with calibration
FP8 (E4M3)	8	Medium (4-bit exp)	Hopper (H100)+	Training & inference

Format	Per-Element Bits	Dynamic Range	Hardware Support	Use Case
INT4 / NF4	4	Very narrow (16 values)	Specialized (GPTQ, AWQ)	Aggressive weight compression
MXINT4 (g=32)	4.25 effective	Moderate (shared exp)	OCP standard	Extreme compression with group scaling

7

Custom Kernel Case Study: MXINT8 Dequantization

7.1 Motivation & Design

The MXINT8 dequantization kernel is the core enabler of weight-only quantization. Its job is to convert weight vectors stored in compact MXINT8 format to BFloat16 for computation. The workflow is:

- **Storage:** Weights reside in MXINT8 in HBM, saving ~4× memory vs. FP32.
- **On-the-fly dequantization:** When a layer needs its weights, the kernel converts them to BFloat16.
- **Compute:** The matmul proceeds in BFloat16 (using Tensor Cores).
- **Discard:** The BFloat16 copies are discarded after the layer completes—they are never stored persistently.

This approach trades a small amount of compute (the dequantization) for a large memory savings. Since model weights dominate memory in large language models, this trade-off is almost always worthwhile.

7.2 CPU Reference Implementation

The CPU kernel performs per-element bit manipulation to convert each MXINT8 mantissa to BFloat16. This implementation is important as a correctness reference and helps understand the bit-level details.

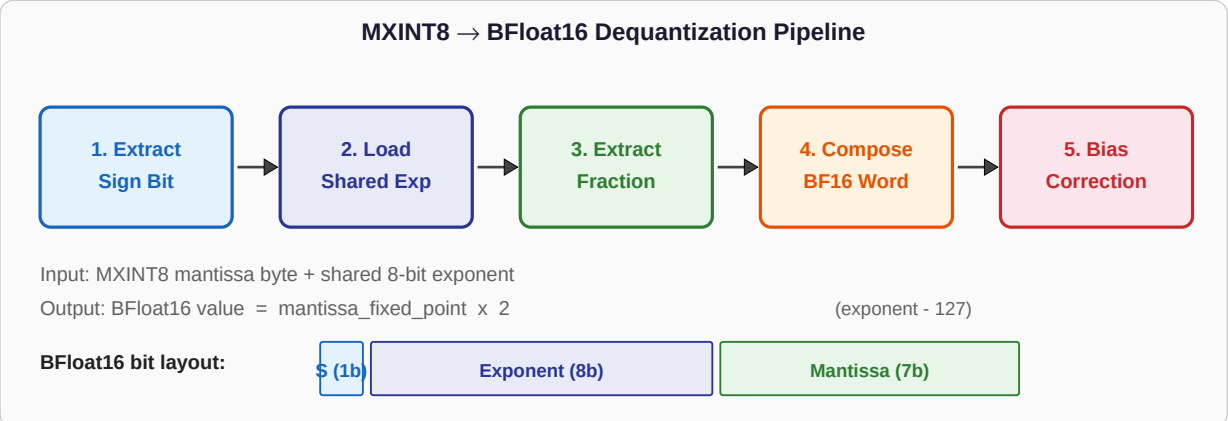

Figure 7.1: MXINT8 to BFloat16 dequantization pipeline

7.2.1 Step-by-Step Bit Manipulation

For each element, the kernel must map an MXINT8 mantissa byte and a shared exponent byte into a 16-bit BFloat16 word. BFloat16 has the layout: [1 sign | 8 exponent | 7 mantissa].

Step 1: Extract Sign Bit

Isolate bit 7 of the MXINT8 mantissa and position it at BFloat16's sign location (bit 15).

```
uint8_t mantissa = 0b11010000;  // = -1.25
uint16_t sign = (mantissa & 0x80) << 8;
// 0x80 = 0b10000000 (mask bit 7)
// sign = 0b10000000_00000000 = 0x8000
// This places the sign at bit 15 of the BF16 word
```

Step 2: Load Shared Exponent

The exponent byte is shared across the group. Index into the exponent array at `[i / group_size]` and shift to BFloat16's exponent field (bits 7–14).

```
uint8_t exp_byte = exponents[i / 32];  // e.g., 130
uint16_t exp_field = (uint16_t)exp_byte << 7;
// 130 = 0b10000010
// exp_field = 0b01000001_00000000 = bits 7-14 of BF16
```

Step 3: Extract Fraction Bits

Take the absolute value of the mantissa, mask the lower 6 bits (the fraction), and shift to BFloat16's mantissa field (bits 0–6).

```
uint8_t abs_mant = mantissa & 0x7F;  // clear sign bit
uint16_t frac = (abs_mant & 0x3F) << 1;
// 0x3F = 0b00111111 (lower 6 bits)
// Shift left 1 to align with BF16 mantissa field
```

Step 4: Compose BFloat16 Word

Bitwise OR the three fields together to form the raw BFloat16 bit pattern.

```
uint16_t bf16_bits = sign | exp_field | frac;
```

7.2.2 The Bias Correction (Implicit Leading 1)

This is the trickiest part of the dequantization. In IEEE 754 (and BFloat16), normalized floating-point numbers have an **implicit leading 1** in the mantissa. That is, the stored mantissa bits represent the *fractional part* of 1.fraction × 2^{exp}. But MXINT's mantissa stores the integer bit **explicitly**.

The Implicit-1 Problem Explained

Case A: Integer bit = 1 (mantissa = `01.101000` = +1.625)
BFloat16 interpretation: 1.{fraction} × 2^{exp}
The implicit leading 1 matches the explicit integer bit → **no correction needed**
Case B: Integer bit = 0 (mantissa = `00.101000` = +0.625)
BFloat16 interpretation: 1.{fraction} × 2^{exp}
BFloat16 thinks the value is 1.101 × 2^{exp} = 1.625 × scale
But the actual MXINT value is 0.625 × scale
The implicit 1 adds an unwanted +1.0 × scale to the result
Fix: When the integer bit is 0, compute `bias = 1.0 × 2^(exp-127)` with the correct sign, and subtract it from the result.
In code: `dont_need_abs = (mantissa & 0x40) != 0`
If `dont_need_abs` is false, apply the bias correction.

Worked Example: Full Dequantization with Bias Correction

Input: mantissa byte = `0b 00101000`, exponent = 130
1. Sign: bit 7 = 0 → positive
2. Integer bit (bit 6) = 0 → bias correction needed
3. Mantissa value: `0.101000` = 0.5 + 0.125 = 0.625
4. Scale = $2^{(130-127)}$ = 8
5. Expected result: +0.625 × 8 = 5.0
Without correction: BFloat16 interprets as 1.{101000} × 2^{exp}
= 1.625 × 8 = 13.0 ✘ (wrong!)
Bias = 1.0 × 8 = 8.0
Corrected = 13.0 - 8.0 = **5.0** ✔

7.3 GPU Kernel Design (CUDA + CUTE)

The CUDA kernel parallelizes dequantization across thousands of GPU threads using NVIDIA's CUTE (CuTe Tensor Expressions) layout abstractions from the CUTLASS library. CUTE provides a way to express complex data layouts and thread-to-data mappings without manual index arithmetic.

7.3.1 Tiling Strategy

The kernel uses a two-level tiling scheme to map the 1D input array to CUDA threadblocks:

• **group_tiler:** Reshapes the 1D input array into a logical 2D matrix of shape [num_groups, group_size]. Each row corresponds to one MXINT group (32 elements sharing one exponent). This ensures that all elements processed by a tile share the same exponent, simplifying the kernel logic.

• **cta_tiler:** Defines how many groups and elements each CUDA threadblock (CTA) processes. Uses CUTE's `local_tile` to assign each CTA its portion of the data based on `blockIdx`.

• **layout_sX:** Defines the thread → data mapping within a CTA. Uses CUTE's `local_partition` to assign each thread its subset of elements.

7.3.2 Shared Memory Staging

MXINT8 data is loaded from global memory to shared memory in **coalesced patterns** (adjacent threads load adjacent bytes), then threads process their assigned elements from shared memory. This is critical because:

- Global memory loads are coalesced (efficient, ~128-byte transactions).
- Shared memory access during the bit-manipulation phase can be arbitrary (no coalescing requirement).
- The shared exponent for each group only needs to be loaded once and broadcast to all 32 threads processing that group.

7.3.3 Predication for Edge Tiles

When the total number of elements is not a perfect multiple of the tile size, the final tile will extend beyond the valid data range. **Predication masks** prevent threads from reading or writing out-of-bounds memory addresses. In CUTE, this is expressed as a boolean mask tensor that gates memory operations.

> **Example: Predication Logic**
>
> If the data has 1000 elements and the tile processes 256 elements at a time:
> • Tiles 0–2: process elements [0–255], [256–511], [512–767] → all valid
> • Tile 3: assigned [768–1023] but only [768–999] are valid
> • Threads assigned to elements 1000–1023 have their predicate set to `false`
> • These threads skip both load and store operations

7.4 GPU vs. CPU Performance Analysis

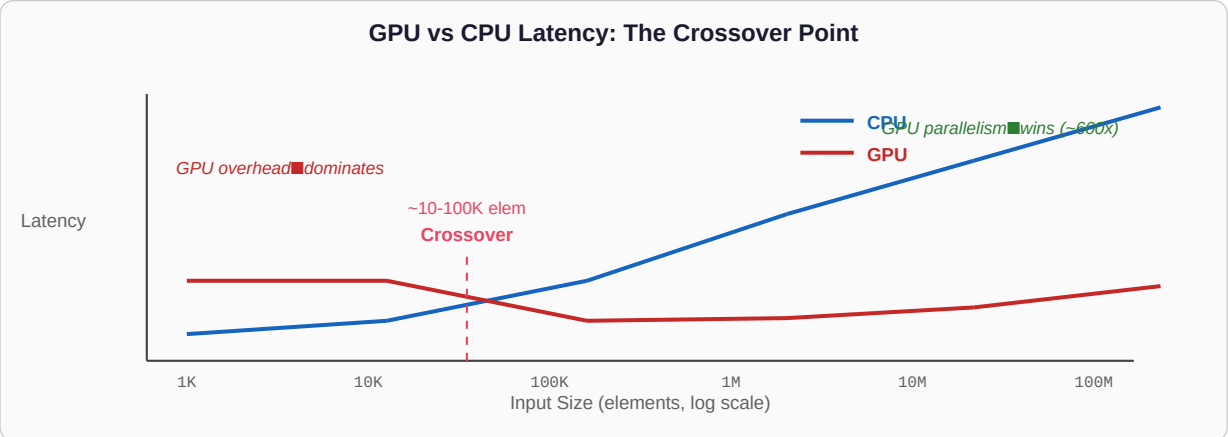

Figure 7.2: GPU vs CPU latency crossover for MXINT8 dequantization

Elements	CPU Latency	GPU Latency	GPU Speedup	Analysis
1,024	~4 µs	~11 µs	0.35×	GPU launch overhead (~10 µs) dominates the trivial computation
2M	~10 ms	~16 µs	~600×	GPU parallelism fully saturated; thousands of threads active

Elements	CPU Latency	GPU Latency	GPU Speedup	Analysis
235M	~1.07 s	~1.7 ms	~620×	Both saturated; bandwidth-limited (data transfer, not compute)

7.4.1 Understanding the Crossover

For small inputs (<~10K elements), the GPU's ~10 µs launch overhead exceeds the total CPU compute time, making the CPU faster. Above ~100K elements, the GPU's massive parallelism (thousands of threads) overtakes the sequential CPU by orders of magnitude.

The speedup plateaus around ~600× because both CPU and GPU become **bandwidth-limited**: the dequantization operation is simple (few FLOPs per element), so performance is bounded by how fast data can be read from and written to memory. The GPU has ~1–3 TB/s HBM bandwidth vs. the CPU's ~50–100 GB/s DDR bandwidth, yielding a ~15–30× raw bandwidth advantage. The remaining speedup comes from the GPU's ability to saturate its bandwidth via massive thread-level parallelism.

7.4.2 Why the Speedup Plateaus

The dequantization kernel performs roughly 10 integer/bitwise operations per element but must read 1 byte (MXINT8 mantissa) + amortized ~0.25 bytes (shared exponent per 32 elements) and write 2 bytes (BFloat16 output). This gives an arithmetic intensity of approximately 10 ops / 3.25 bytes ≈ 3 ops/byte—far below the GPU's ridge point of ~156 FLOPS/byte. The kernel is **severely memory-bound**, and the bottleneck is HBM bandwidth, not compute.

> **Practical Takeaway**
>
> For weight-only quantization in a real model, the dequantization kernel processes millions of weights per layer. At these sizes, the GPU delivers ~600× speedup over CPU, and the dequantization cost is negligible compared to the subsequent matrix multiplication. The memory savings (3.88× compression) far outweigh the tiny compute overhead.

7.5 Integration: End-to-End Model Quantization

The dequantization kernel enables the QLinearPacked module, which replaces standard `nn.Linear` layers. During the forward pass, QLinearPacked:

- Calls the MXINT8 dequantization kernel to convert weights from MXINT8 to BFloat16.
- Performs the standard matrix multiplication in BFloat16 using Tensor Cores.
- Returns the result; the temporary BFloat16 weights are freed.

In the Lab 4 DeBERTa-XLarge experiment (766M parameters), this approach achieved:

Metric	FP32 Baseline	MXINT8 Quantized	Change
Peak GPU Memory	2,906 MB	976 MB	–66.4%
Prediction	joy	joy (correct)	Match

Metric	FP32 Baseline	MXINT8 Quantized	Change
Top-1 Logit	7.3452	7.3502	<0.01 deviation

Why 66.4% Reduction and Not the Theoretical 74.2%?

Theoretical max: (32 - 8.25) / 32 = 74.2% compression from FP32 to MXINT8 (g=32).
The gap is explained by several factors:

- **Classifier head excluded:** The final classification layer remains in FP32
- **Activations in full precision:** Only weights are quantized; intermediate activations remain in FP32/BF16
- **Non-quantized parameters:** Embeddings, LayerNorm weights, and biases stay in FP32
- **Temporary dequantized weights:** BFloat16 copies temporarily occupy GPU memory during each layer's computation
- **Framework overhead:** PyTorch CUDA context, memory allocator metadata, etc.